

Quantum-Assisted Harnesses for Verifiable AI Iteration: Candidate Search via Amplitude Amplification and Annealing

Mojofull Furoku
mojofull.furoku@gmail.com

April 26, 2026

Abstract

Large language model agents often solve tasks through iterative planning, tool use, generation, testing, and repair. When each step has a nonzero failure probability, long-horizon tasks suffer from accumulated failure and from the cost of repeated attempts. This paper studies a narrow but technically useful reformulation: treat an AI agent iteration as a search over candidate trajectories $x \in X$, where a mechanical verifier $V(x)$ marks success. Under ideal coherent-oracle assumptions, Grover search and amplitude amplification reduce the verifier-query cost of finding a successful candidate from $O(1/p)$ for random retry to $O(1/\sqrt{p})$, where p is the success mass of the candidate generator. When success criteria can be expressed as penalties, the same framework can be mapped to QUBO or Ising objectives for annealing-based search. We provide a reproducible CPU-only experimental harness: state-vector Grover simulation, random-retry baselines, a structured QUBO verifier, classical simulated annealing, and a small closed-system quantum-annealing simulation. For $N = 65,536$ candidates with one success, the classical random-retry expectation is 65,536 verifier calls, whereas the Grover simulation reaches success probability 0.999988 with an expected 202.0 verifier/oracle calls. In an 18-bit QUBO verifier with 76 zero-energy states among 262,144 candidates, random retry requires an expected 3,449.3 attempts, while classical simulated annealing finds zero-energy candidates in all 500 runs. In an 8-bit closed-system annealing simulation, final ground-state probability increases from 0.0285 to 0.6025 as anneal time increases from 0.5 to 8.0. These results do not demonstrate quantum advantage on current hardware. They instead identify the conditions under which quantum search primitives can enter AI harness design: the task must be discretized, mechanically verifiable, and resistant to verifier hacking; coherent implementation and hardware constraints remain major barriers.

1 Introduction

AI agents built on large language models (LLMs) increasingly solve tasks by iterating: plan, call tools, generate artifacts, test, repair, and try again. This improves capability but also exposes an accumulation problem. If a task requires L fragile steps and each step succeeds with probability q , a naive independent model gives a whole-task success rate of approximately q^L . Even with $q = 0.95$, one hundred steps yield only about 0.0059 probability of completing without error. Real agents are not independent Bernoulli chains, but the example captures a practical point: long-horizon agent workflows need scaffolding, testing, rollback, and verification rather than unconstrained sampling.

Current harness engineering reduces failure by surrounding the model with external structure: instructions, tool interfaces, state management, unit tests, validators, and retry policies. Anthropic’s Skills framework, for example, packages task-specific instructions, scripts, and resources to be loaded only when needed [2]. Software-engineering benchmarks such as SWE-bench also illustrate

the importance of mechanically checkable outcomes: a candidate patch is not merely judged by plausibility but by whether it resolves a real issue under tests [10, 12, 6].

This paper asks whether quantum search primitives could reduce the cost of reliable AI iteration once such mechanical verification exists. The claim is not that quantum computers eliminate AI errors, nor that current quantum hardware can run an LLM or a large agent loop. The narrower claim is that part of the agent loop can sometimes be separated into a verifiable candidate search problem. If a generator produces candidates with success probability p and a verifier can mark success, then Grover search and amplitude amplification offer a quadratic query improvement in an ideal oracle model [8, 3]. If success is expressible as an energy or penalty function, QUBO and Ising formulations provide a route to annealing-style optimization [11, 4, 5].

The contributions are:

1. A formal mapping from AI agent iteration to verifiable candidate search.
2. A separation between ideal amplitude amplification, QUBO/annealing formulations, and current-hardware limitations.
3. A reproducible CPU experiment suite that checks the scaling laws and produces concrete baselines.
4. A research roadmap for moving from classical harnesses to small quantum or hybrid demonstrations.

2 Problem formulation

Let X be a finite set of candidate agent trajectories. A candidate may represent a code patch, a tool-call sequence, a proof sketch, a plan, or a discrete schedule. Let

$$V : X \rightarrow \{0, 1\} \tag{1}$$

be a mechanical verifier, where $V(x) = 1$ means the candidate passes the task’s acceptance criteria. Denote

$$M = |\{x \in X : V(x) = 1\}|, \quad N = |X|, \quad p = M/N \tag{2}$$

for uniform sampling. More generally, if a candidate generator A induces a distribution over X , p is the probability mass assigned to successful candidates.

Classical random retry samples candidates until $V(x) = 1$. Under independent samples, the expected number of verifier calls is

$$\mathbb{E}[C_{\text{classical}}] = \frac{1}{p}. \tag{3}$$

This is exactly the failure-accumulation bottleneck for rare successes: if the verifier is expensive or the candidate space is large, repeated attempts dominate cost.

In a quantum setting, suppose an algorithm A prepares

$$A|0\rangle = \sqrt{a} |\psi_{\text{good}}\rangle + \sqrt{1-a} |\psi_{\text{bad}}\rangle, \tag{4}$$

where a is the success probability under the prepared distribution. If V can be implemented coherently as an oracle that phase-flips good states, amplitude amplification obtains a good state using $O(1/\sqrt{a})$ applications of A , A^{-1} , and the oracle [3]. In the uniform case, this becomes the standard $O(\sqrt{N/M})$ Grover scaling [8]. Fixed-point variants can reduce overshoot behavior when success fractions are uncertain, at the price of different constants and assumptions [13].

Table 1: Mapping between AI harness concepts and quantum-search concepts.

AI harness concept	Search formulation	Quantum-search analogue
Candidate plan, patch, or tool trace	$x \in X$	basis state or superposition
Unit test, type check, proof checker	verifier $V(x)$	oracle / phase flip
Repeated attempts	random sampling	$O(1/p)$ query cost
Success amplification	biased candidate selection	amplitude amplification
Constraint penalties	energy $E(x)$	QUBO / Ising Hamiltonian
Best verified candidate	low-energy or good state	annealing sample / measurement

The same verifier can sometimes be represented as an energy function $E : X \rightarrow \mathbb{R}_{\geq 0}$, where lower energy means fewer violated constraints and $E(x) = 0$ denotes success. If x is encoded as bits, a quadratic unconstrained binary optimization (QUBO) objective

$$E(x) = x^T Q x + c^T x + \text{const.} \quad (5)$$

can be mapped to an Ising Hamiltonian. Annealing-based methods then attempt to sample low-energy states. Adiabatic quantum computation and quantum annealing are natural theoretical and engineering settings for this approach [7, 1, 5].

Table 1 summarizes the mapping.

3 Which AI tasks fit the formulation?

The formulation is useful only when success can be checked mechanically. Suitable examples include:

- code generation with unit tests, integration tests, type checking, or static analysis;
- theorem proving when a proof checker validates the final artifact;
- scheduling, routing, and resource allocation with explicit constraints;
- bounded design search with measurable cost and constraint violations;
- agent plans whose goal state can be simulated or formally checked.

The formulation is weaker for tasks where success is subjective or delayed: essay quality, product strategy, user satisfaction, or scientific hypotheses whose truth is established only later. A learned reward model can be used as a proxy verifier, but then the system inherits reward-model errors and verifier hacking risks. Thus, the core bottleneck is not just quantum hardware. It is the design of reliable verifiers.

4 Algorithms

4.1 Grover and amplitude-amplification harness

The idealized harness is:

1. Encode candidates $x \in X$ as bit strings.
2. Prepare a candidate distribution with a unitary A .

3. Implement $V(x)$ as a coherent phase oracle.
4. Apply amplitude amplification.
5. Measure a candidate.
6. Re-check the measured candidate with the ordinary classical verifier.

The final classical re-check is important. In a realistic AI setting, the quantum procedure is a candidate-selection mechanism, not a replacement for acceptance testing. This paper counts r Grover oracle calls plus one final verification call, and divides by the measured success probability to obtain an expected verifier/oracle-call estimate.

4.2 QUBO and annealing harness

For constraint-structured tasks, we define penalties $P_i(x)$ and energy

$$E(x) = \sum_i \lambda_i P_i(x), \quad (6)$$

where $\lambda_i > 0$ weights hard or soft constraints. Examples include ordering constraints (“build before test”), exclusivity constraints (“do not select incompatible actions”), and cardinality constraints (“select exactly k validation actions”). If these penalties are quadratic in binary variables, they form a QUBO. If not, auxiliary variables or higher-order reductions may be required [11].

Annealing is not a guarantee of global optimality. It is a structured sampler biased toward low-energy states. Therefore, it must be evaluated against classical optimizers such as greedy local search, simulated annealing, SAT/SMT solvers, integer programming, and Monte Carlo tree search.

5 Experimental method

All experiments are CPU-only and implemented in one Python script, `quantum_agent_search_experiment.py`. The script uses NumPy, SciPy, and Matplotlib. It does not use a quantum SDK or quantum hardware. The random seed is 20260425. Source code and frozen outputs are included as ancillary files.

5.1 Experiment A: state-vector Grover simulation

For $n \in \{8, 10, 12, 14, 16\}$ qubits and success counts $M \in \{1, 4, 16\}$, the script selects a random success set and simulates Grover iterations explicitly. The optimal iteration count is

$$r = \text{round} \left(\frac{\pi}{4 \arcsin \sqrt{M/N}} - \frac{1}{2} \right). \quad (7)$$

The simulation is checked against the analytic success probability

$$P_{\text{succ}}(r) = \sin^2 \left((2r + 1) \arcsin \sqrt{M/N} \right). \quad (8)$$

Classical random retry is sampled from a geometric distribution with probability $p = M/N$.

5.2 Experiment B: QUBO verifier and classical simulated annealing

The 18-bit toy verifier models an agent trajectory with fixed plan bits, selected action bits, and validation bits. Zero energy means all constraints pass:

- bits 0–5 equal the prefix $[1, 0, 1, 1, 0, 0]$;
- bits 6–11 have Hamming weight 2;
- bits 12–17 have Hamming weight 3;
- bits 6 and 12 match, and bits 7 and 13 match.

The energy is the sum of squared violations. The script exhaustively enumerates all 2^{18} states to determine the true ground-state count, then runs 500 classical simulated-annealing trials with bit-flip proposals and an exponential temperature schedule.

5.3 Experiment C: small closed-system quantum annealing simulation

The 8-bit annealing experiment constructs

$$H(s) = (1 - s)H_0 + sH_P, \quad (9)$$

where $H_0 = -\sum_i X_i$ is a transverse-field driver and H_P is diagonal with the 8-bit QUBO energy. The initial state is the uniform superposition. Time evolution is approximated using SciPy’s `expm_multiply`. The experiment varies anneal time $T \in \{0.5, 1, 2, 4, 8\}$ with 160 time steps and measures final ground-state probability.

5.4 Unit tests

The script includes four unit tests:

1. state-vector Grover probability matches the analytic expression;
2. the optimal Grover iteration count gives success probability above 0.95 for $N = 256, M = 1$;
3. zero-energy 18-bit QUBO states satisfy the verifier constraints;
4. the closed-system annealing simulation preserves state norm.

All four tests pass.

6 Results

6.1 Grover query scaling

Table 2 reports the single-success case. The expected classical retry cost grows as N , while the Grover verifier/oracle-call estimate grows approximately as $\sqrt{N}$.

With $N = 65,536$ and $M = 1$, random retry requires an expected 65,536 verifier calls. The Grover simulation uses $r = 201$ iterations, reaches success probability 0.999988, and yields an expected 202.0 verifier/oracle calls. This is a query reduction factor of approximately 324.4.

Table 3 shows that multiple-success cases follow the same N/M versus $\sqrt{N/M}$ pattern.

Table 2: Single-success search ($M = 1$). The quantum column counts r Grover oracle calls plus one final verification call, divided by simulated success probability.

n	N	Classical exp.	r	Grover P_{succ}	Quantum exp.
8	256	256.0	12	0.999947	13.001
10	1,024	1,024.0	25	0.999461	26.014
12	4,096	4,096.0	50	0.999945	51.003
14	16,384	16,384.0	100	1.000000	101.000
16	65,536	65,536.0	201	0.999988	202.002

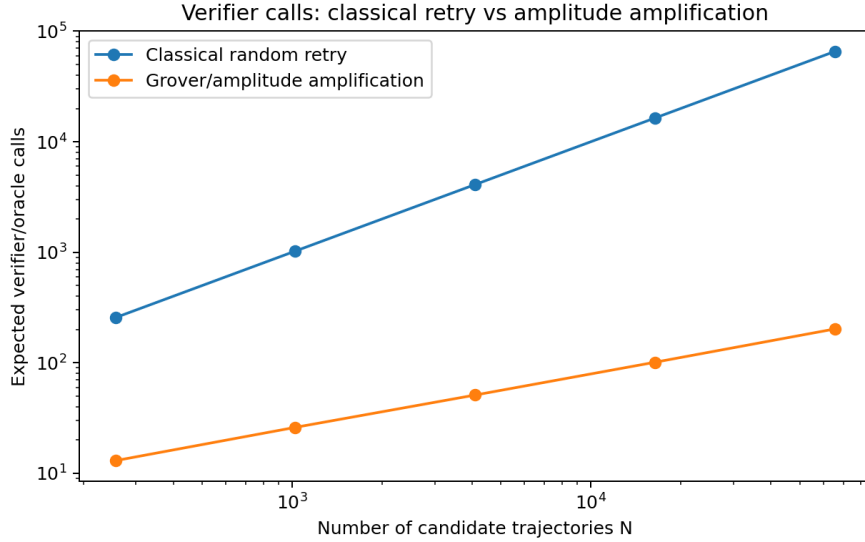


Figure 1: Expected verifier/oracle calls for classical random retry and Grover/amplitude amplification in the single-success case.

6.2 QUBO verifier and simulated annealing

The 18-bit QUBO has 76 zero-energy states among 262,144 candidates. The random success fraction is 0.0002899, giving an expected 3,449.3 random attempts. Classical simulated annealing finds a zero-energy state in all 500 runs, with median 293 steps to first success. Table 4 reports the result.

This experiment confirms a practical point for AI harness design: if acceptance criteria can be expressed as constraints, candidate search can exploit structure rather than relying on blind retry. Whether a quantum annealer improves on strong classical baselines is a separate empirical question.

6.3 Closed-system quantum annealing simulation

Table 5 reports the 8-bit annealing simulation. Increasing anneal time increases final ground-state probability and decreases expected energy.

The result is consistent with the adiabatic intuition that slower schedules can concentrate more probability on low-energy states. It does not predict large-scale hardware performance.

Table 3: Multiple-success search ($M = 16$).

n	N	Classical exp.	Quantum exp.	Reduction factor
8	256	16.0	4.161	3.85
10	1,024	64.0	7.024	9.11
12	4,096	256.0	13.001	19.69
14	16,384	1,024.0	26.014	39.36
16	65,536	4,096.0	51.003	80.31

Table 4: Structured 18-bit QUBO verifier. Simulated annealing is a classical baseline, not evidence of quantum advantage.

Metric	Value
Candidate bits	18
Candidates	262,144
Zero-energy states	76
Random success fraction	0.0002899
Random expected attempts	3,449.3
Simulated-annealing runs	500
Simulated-annealing success rate	1.000
Median steps to success	293
Mean best energy	0.000

7 Discussion

The experiments establish three limited claims. First, when an AI iteration is abstracted as verifiable candidate search, the amplitude-amplification query law appears directly in simulation. Second, when success can be decomposed into constraint penalties, a verifier can be made into an objective function that structured search methods can exploit. Third, closed-system annealing dynamics can connect the same objective to a quantum-mechanical low-energy search model.

The experiments do not establish that present-day quantum computers accelerate LLM agents. Several barriers remain. The verifier must be coherent for amplitude amplification, requiring reversible quantum implementation of the relevant predicate. Current LLM inference is a large classical computation, and naively embedding it into a coherent oracle is not realistic. Near-term use is more likely to involve a classical LLM generating a bounded candidate set or constraint model, followed by classical or quantum optimization over that reduced problem.

Verifier quality is also central. A verifier can be incomplete, wrong, or gameable. Unit tests can miss bugs; proof obligations can omit assumptions; reward models can be exploited. Quantum search amplifies whatever the verifier labels as good. It therefore magnifies verifier design mistakes as well as verifier successes.

Finally, annealing has no general guarantee of finding global optima efficiently. Hardware implementations face noise, temperature, coefficient precision, minor embedding, chain breaks, calibration drift, and readout error. Gate-model amplitude amplification faces circuit-depth and fault-tolerance requirements. IBM Quantum’s educational material similarly emphasizes that many major quantum algorithms are theoretically important but require error-corrected hardware for large-scale deployment [9].

Table 5: Small closed-system quantum annealing simulation. The model excludes hardware noise, embedding overhead, finite-temperature effects, and control error.

Anneal time T	Ground prob.	Expected energy	Norm error
0.5	0.028523	2.824789	2.22×10^{-16}
1.0	0.045663	2.420634	2.22×10^{-16}
2.0	0.119312	1.712687	0.00
4.0	0.310914	1.017029	0.00
8.0	0.602547	0.468370	0.00

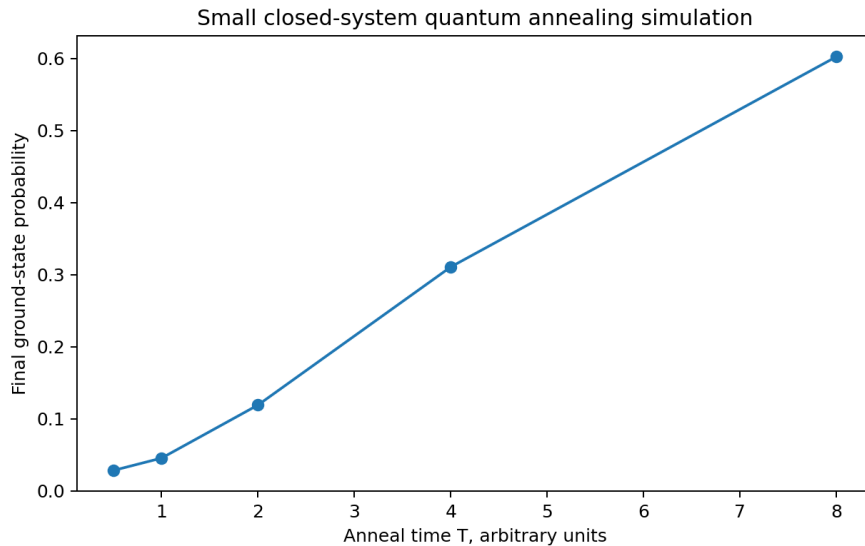


Figure 2: Final ground-state probability in the small closed-system annealing simulation.

8 Research roadmap

A realistic research path has four stages.

Stage 1: classical verifiable harnesses. Collect tasks with mechanical verifiers, including coding tasks, proof-checking tasks, and scheduling problems. Measure pass@k, verifier cost, false positives, and false negatives. Compare random retry, beam search, Monte Carlo tree search, SAT/SMT, integer programming, and simulated annealing.

Stage 2: quantum simulation. Build small coherent verifier circuits where possible. For constraint tasks, translate penalties to QUBO or Ising form. Compare state-vector Grover, fixed-point search, tensor-network simulation, and closed-system annealing against classical baselines.

Stage 3: small hardware demonstrations. Run tiny Grover or amplitude-amplification examples on gate-model devices and tiny QUBOs on annealing hardware. Measure shot counts, circuit depth, embedding overhead, noise, and wall-clock cost, not only query complexity.

Stage 4: hybrid AI agents. Use LLMs to propose candidate spaces, constraints, and repair operations. Use optimizers to search within the generated discrete structure. Always finish with an independent classical verifier and feed failures back into the constraint model.

9 Limitations and risks

Success-oracle construction. Many AI tasks lack reliable binary success predicates. This framework is strongest for tasks where an independently engineered verifier exists.

Coherent oracle construction. Amplitude amplification requires coherent A , A^{-1} , and V . This is much stronger than having an ordinary Python test suite.

Verifier hacking. If the verifier is incomplete, the search process may find candidates that pass the verifier while failing the user’s true goal.

Benchmark overfitting. Public benchmarks can become contaminated or over-optimized. Verifiable harness research should use held-out, dynamic, or generated tasks when possible.

Cost accounting. Query complexity can hide state-preparation, compilation, error-correction, and I/O costs. Any future claim of advantage must report end-to-end wall-clock and energy costs against strong classical baselines.

10 Conclusion

Quantum computation does not remove AI failure by magic. However, if an AI task can be transformed into a mechanically verifiable candidate search problem, quantum search primitives provide a clear place to enter the harness: amplitude amplification can quadratically reduce verifier-query cost in an ideal oracle model, and QUBO/Ising formulations can connect structured verification to annealing-style optimization.

The reproducible CPU experiments support the formulation rather than a hardware-advantage claim. In the largest Grover simulation, the expected query count falls from 65,536 random verifier calls to approximately 202 verifier/oracle calls under ideal amplitude amplification. In the QUBO experiment, constraint structure makes blind retry unnecessary. In the small annealing simulation, longer anneal times increase ground-state probability.

The central open problem is therefore not merely obtaining more qubits. It is designing AI tasks, harnesses, and verifiers that are discrete, reliable, non-gameable, and cheap enough to serve as search oracles or energy functions.

Reproducibility and code availability

The ancillary files include the Python experiment script, JSON/CSV outputs, and a requirements file. To reproduce the experiments, run:

```
python3 quantum_agent_search_experiment.py --out results --test
```

The expected unit-test summary is four tests run with zero failures and zero errors.

AI assistance disclosure

The author used GPT-5.5 Pro as an assistive tool for manuscript drafting, editing, code organization, and packaging. The author reviewed the manuscript, source code, numerical results, references, and arXiv metadata, and takes full responsibility for the final submitted content.

Acknowledgments

No external funding or additional acknowledgments are reported.

References

- [1] Tameem Albash and Daniel A. Lidar. Adiabatic quantum computation. *Reviews of Modern Physics*, 90:015002, 2018.
- [2] Anthropic. The complete guide to building skills for claude. Anthropic Resources, 2026. Accessed 2026-04-25.
- [3] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Contemporary Mathematics*, 305:53–74, 2002.
- [4] D-Wave Systems. Qubos and ising models. D-Wave Documentation, 2026. Accessed 2026-04-25.
- [5] D-Wave Systems. What is quantum annealing? D-Wave Documentation, 2026. Accessed 2026-04-25.
- [6] X. Deng et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025.
- [7] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. Quantum computation by adiabatic evolution. *arXiv preprint quant-ph/0001106*, 2000.
- [8] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [9] IBM Quantum. Which problems are quantum computers good for? IBM Quantum Learning, 2026. Accessed 2026-04-25.
- [10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023.
- [11] Andrew Lucas. Ising formulations of many np problems. *Frontiers in Physics*, 2:5, 2014.
- [12] OpenAI. Introducing swe-bench verified, 2024. Accessed 2026-04-25.
- [13] Theodore J. Yoder, Guang Hao Low, and Isaac L. Chuang. Fixed-point quantum search with an optimal number of queries. *Physical Review Letters*, 113:210501, 2014.